GENERATIVE AI & LLMS

Retrieval-Augmented Generation (RAG)

Ground large language models in your own data with retrieval, reranking, and rigorous evaluation.

RAG · LLM · embeddings · vector-search · retrieval · reranking · evaluation

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. 1. What RAG Is and Why It Matters
2. 2. Getting Started
3. 3. The Anatomy of a RAG Pipeline
4. 4. Chunking Strategies
5. 5. Embeddings and the Vector Space
6. 6. Indexing and Approximate Nearest Neighbour Search
7. 7. Hybrid Search: Dense + Sparse
8. 8. Reranking
9. 9. Generation: Prompting and Citations
10. 10. Evaluating a RAG System
11. 11. Common Failure Modes
12. 12. Advanced RAG Patterns
13. 13. Cheat-Sheet Reference
14. 14. Best Practices and Pitfalls
15. 15. Interview Questions & Answers
16. 16. Further Learning

Retrieval-Augmented Generation (RAG) turns a general-purpose language model into a domain expert by feeding it the *right* passages from your own knowledge base at query time. Instead of hoping the model memorised a fact during training, you retrieve authoritative text and let the model reason over it — cutting hallucinations, enabling citations, and keeping answers fresh without retraining.

1. What RAG Is and Why It Matters

A raw large language model (LLM) is a fixed snapshot: its knowledge stops at its training cutoff, it cannot see your private documents, and when it lacks a fact it will often **confidently invent one**. RAG solves this by splitting the problem into two systems — a **retriever** that finds relevant text and a **generator** (the LLM) that writes the answer grounded in that text.

FRESHNESS

Update the knowledge base, not the model. New docs are searchable in seconds — no fine-tuning, no GPU cluster.

GROUNDING

Answers cite retrieved passages, so users can verify claims and you can measure faithfulness.

COST

Cheaper than fine-tuning for factual knowledge; you pay per-token for context instead of retraining weights.

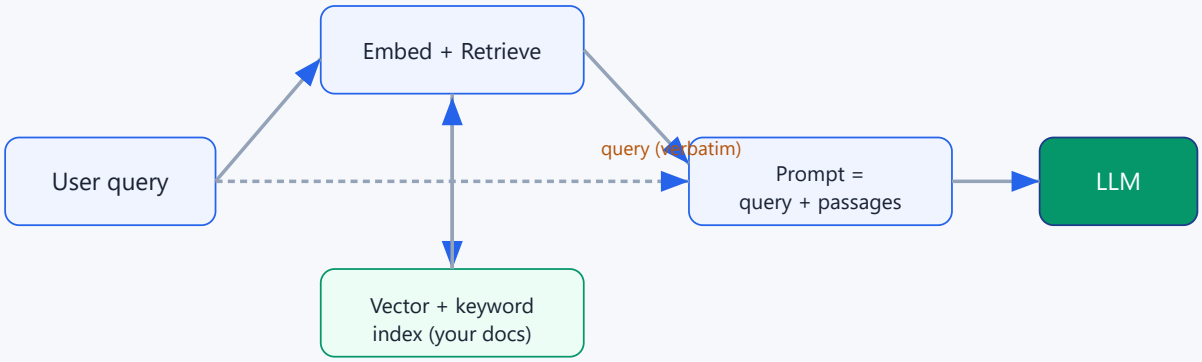


Fig 1 — The RAG loop: the query drives retrieval; retrieved passages plus the original query become the LLM prompt.

Mental model

RAG is "open-book exam" for an LLM. Fine-tuning changes *how* the model thinks; RAG changes *what it can see* right now. Use fine-tuning for behaviour and format, RAG for knowledge and facts.

2. Getting Started

A minimal stack needs three things: an embedding model, a vector store, and an LLM. Here we use open, widely-available components.

BASH

```bash
# Create an isolated environment
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate

# Core libraries
pip install langchain langchain-community langchain-text-splitters
pip install sentence-transformers      # local embeddings
pip install chromadb faiss-cpu         # two popular vector stores
pip install rank-bm25                  # keyword/sparse retrieval for hybrid search
pip install "unstructured[pdf]"        # document loaders
```

PYTHON

```python
# hello_rag.py — a complete 30-line RAG pipeline
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma

# 1) Load & chunk
docs = TextLoader("handbook.txt", encoding="utf-8").load()
chunks = RecursiveCharacterTextSplitter(
    chunk_size=800, chunk_overlap=120
).split_documents(docs)

# 2) Embed & index
emb = HuggingFaceEmbeddings(model_name="BAAI/bge-small-en-v1.5")
store = Chroma.from_documents(chunks, emb, collection_name="hr")

# 3) Retrieve
retriever = store.as_retriever(search_kwargs={"k": 4})
hits = retriever.invoke("How many vacation days do new hires get?")
for h in hits:
    print(h.page_content[:120], " ... ")
```

> **Tip**
>
> Start local (sentence-transformers + Chroma) so you can iterate for free, then swap the embedding and LLM calls for a hosted API once the pipeline shape is proven. The interfaces are identical.

3. The Anatomy of a RAG Pipeline

Production RAG is a pipeline of distinct stages, each independently tunable. Two of them run **offline** (ingestion) and the rest run **online** (per query).

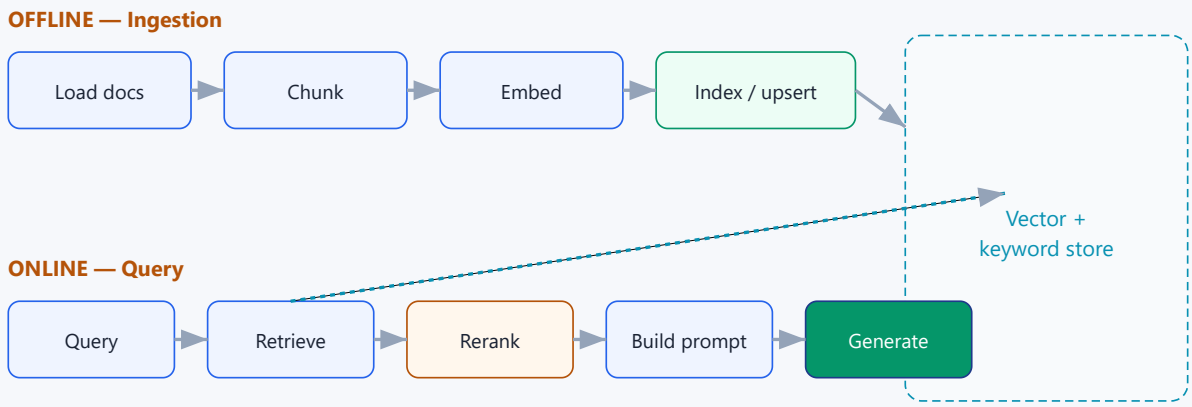


Fig 2 — Offline ingestion builds the index once; the online path retrieves, reranks, and generates per query.

Stage	Responsibility	Key knobs
Load	Parse PDFs, HTML, DB rows into text + metadata	Parser quality, OCR, table extraction
Chunk	Split into retrievable units	Size, overlap, boundaries
Embed	Map text to vectors	Model, dimension, normalisation
Index	Store vectors for fast ANN search	HNSW params, metric
Retrieve	Find top-k candidates	k, hybrid weights, filters
Rerank	Reorder candidates by true relevance	Cross-encoder, top-n kept
Generate	Compose grounded answer	Prompt, temperature, citations

4. Chunking Strategies

Chunking is the most underrated lever in RAG. Chunk too large and you dilute the embedding and waste context tokens; chunk too small and you fragment ideas so no single chunk carries the answer.

FIXED-SIZE + OVERLAP

Split every N characters/tokens with an overlap window (e.g. 800/120). Simple, robust default. Overlap prevents cutting a sentence off from its context.

RECURSIVE / STRUCTURAL

Split on a hierarchy of separators (paragraphs → sentences → words) so boundaries respect structure. LangChain's `RecursiveCharacterTextSplitter`.

SEMANTIC

Embed sentences, start a new chunk when cosine similarity to the running group drops below a threshold. Keeps topically coherent units together.

DOCUMENT-AWARE

Respect Markdown headings, code blocks, or table rows. Attach the heading path as metadata so a chunk knows its section.

PYTHON

```python
# Attach heading context so a retrieved chunk carries its section title
from langchain_text_splitters import MarkdownHeaderTextSplitter, RecursiveCharacterTex

md = MarkdownHeaderTextSplitter(headers_to_split_on=[
    ("#", "h1"), ("##", "h2"), ("###", "h3"),
])
sections = md.split_text(open("guide.md", encoding="utf-8").read())

# Second pass: keep sections under the model's ideal size, preserving metadata
splitter = RecursiveCharacterTextSplitter(chunk_size=700, chunk_overlap=100)
chunks = splitter.split_documents(sections)
# Each chunk.metadata now has {"h1": ..., "h2": ...} — great for filtering & citing
```

Rule of thumb

Target ~200–400 tokens per chunk for dense factual Q&A, with 10–20% overlap. Store rich metadata (source, page, section, timestamp) — it powers filtering, citations, and debugging.

The "parent document" trick

Embed *small* chunks for precise retrieval, but return the *larger* parent passage to the LLM for context. This decouples retrieval granularity from generation context.

5. Embeddings and the Vector Space

An embedding model maps text to a fixed-length vector such that semantically similar text lands nearby. Retrieval is then a nearest-neighbour search under a similarity metric — usually **cosine similarity**.

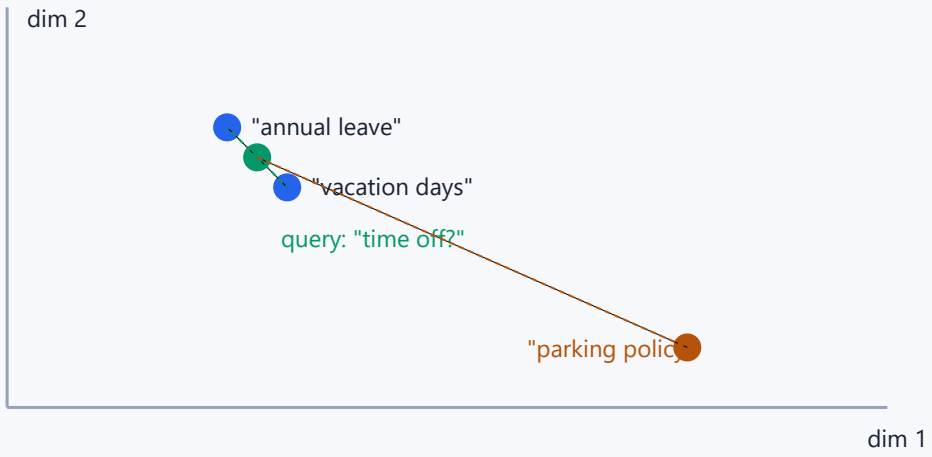


Fig 3 — Semantically related passages cluster near the query vector; unrelated ones sit far away regardless of shared keywords.

PYTHON

```python
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("BAAI/bge-small-en-v1.5")
# normalize_embeddings=True lets us use a plain dot product as cosine similarity
vecs = model.encode(
    ["How much annual leave?", "vacation entitlement", "office parking rules"],
    normalize_embeddings=True,
)
sim = vecs @ vecs.T          # cosine similarity matrix (unit vectors)
print(np.round(sim, 2))
# Row 0 vs 1 (leave ~ vacation) is high; vs 2 (parking) is low.
```

Choice	Trade-off
Dimension (384 vs 1024)	Higher dims capture more nuance but cost more memory & slower search.
Metric (cosine / dot / L2)	Cosine ignores magnitude — the default for normalised text embeddings.
Domain fit	A model tuned on your domain (legal, code, biomedical) beats a bigger general model.
Query vs passage prompts	Many models (e.g. BGE, E5) want an instruction prefix on queries — read the model card.

> ⚠ **Consistency**
> You MUST embed queries and documents with the *same* model and the same normalisation. Mixing models silently destroys retrieval quality — the vector spaces are incompatible.

6. Indexing and Approximate Nearest Neighbour Search

Exact nearest-neighbour search over millions of vectors is too slow. Vector databases use **Approximate Nearest Neighbour (ANN)** indexes — most commonly `HNSW` (Hierarchical Navigable Small World), a layered graph you greedily walk toward the query.

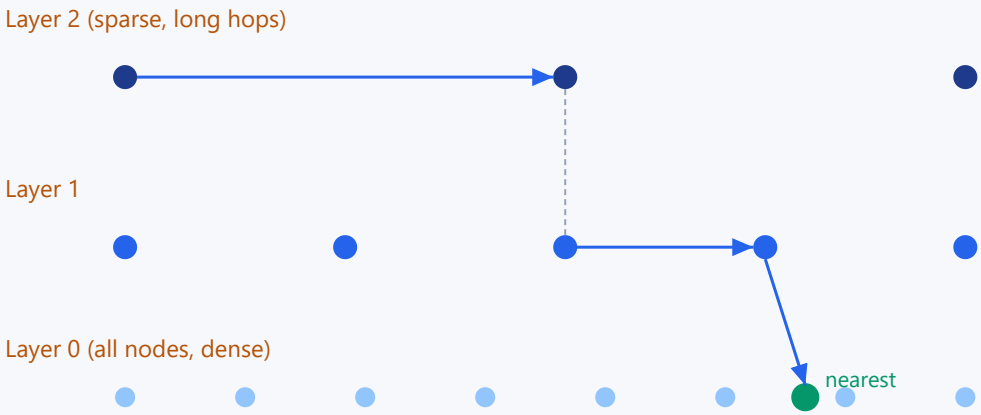


Fig 4 — HNSW: start at the top layer with long hops, descend layer by layer, refining to the true nearest neighbours at the base.

Parameter	Effect
`M`	Edges per node. Higher = better recall, more memory.
`ef_construction`	Build-time search width. Higher = better graph, slower indexing.
`ef_search`	Query-time width. The main recall/latency dial at serve time.

PYTHON

```python
# FAISS: exact vs HNSW, and metadata filtering pattern with Chroma
import faiss, numpy as np
d = 384
xb = np.random.rand(100_000, d).astype("float32")
faiss.normalize_L2(xb)

flat = faiss.IndexFlatIP(d)          # exact, brute force (baseline)
flat.add(xb)

hnsw = faiss.IndexHNSWFlat(d, 32)    # M=32
hnsw.hnsw.efConstruction = 200
hnsw.add(xb)
hnsw.hnsw.efSearch = 64              # tune recall vs latency at query time

q = xb[:1]
D, I = hnsw.search(q, k=5)           # top-5 approximate neighbours
print(I)
```

7. Hybrid Search: Dense + Sparse

Dense (embedding) search excels at meaning but can miss exact identifiers — error codes, SKUs, proper nouns. Sparse keyword search (BM25) nails exact terms but misses paraphrase. **Hybrid search** fuses both, typically via **Reciprocal Rank Fusion (RRF)**.

PYTHON

```python
# Reciprocal Rank Fusion: combine two ranked lists into one robust ranking
def rrf(rank_lists, k=60):
    scores = {}
    for ranking in rank_lists:              # each = [doc_id, doc_id, ...] best-first
        for rank, doc_id in enumerate(ranking):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank + 1)
    return sorted(scores, key=scores.get, reverse=True)

dense = ["d3", "d1", "d7", "d2"]   # from vector search
sparse = ["d1", "d9", "d3", "d5"]  # from BM25
print(rrf([dense, sparse]))        # d1 & d3 rise: they rank well in BOTH systems
```

✓ **Why RRF works**

It needs no score calibration between systems (cosine and BM25 scores are not comparable). It only uses *ranks*, rewarding documents that appear high in multiple lists.

8. Reranking

Retrieval optimises for *recall* (don't miss the right passage) by casting a wide net (k=20–50). A **reranker** then optimises for *precision*, scoring each candidate against the query with a heavier, more accurate model, and keeps the top 3–5.

The key distinction: a **bi-encoder** (your embedding model) encodes query and document *separately* — fast, cacheable, but coarse. A **cross-encoder** feeds query and document *together* into a transformer, attending across both — far more accurate, but too slow to run over the whole corpus. So you use bi-encoder to retrieve and cross-encoder to rerank.

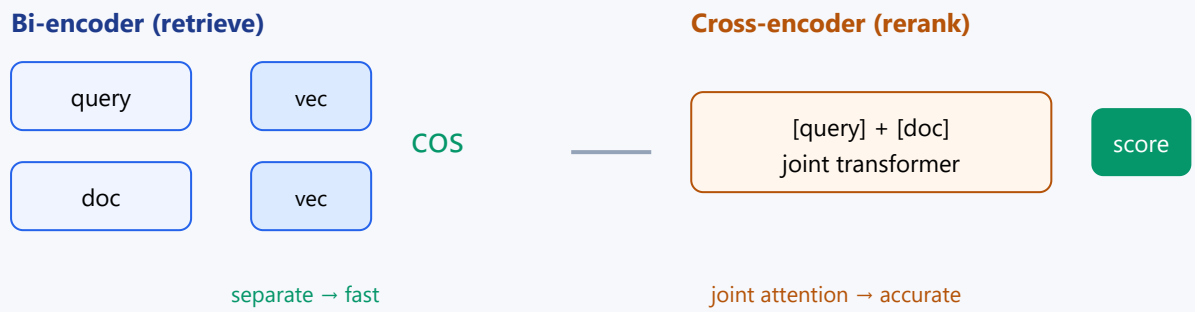


Fig 5 — Bi-encoders scale to millions of docs; cross-encoders re-score a short candidate list with full cross-attention.

PYTHON

```python
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
query = "How many vacation days do new hires get?"
candidates = [c.page_content for c in retriever.invoke(query)]  # k=20 recall net

scored = reranker.predict([(query, c) for c in candidates])
top = [c for _, c in sorted(zip(scored, candidates), reverse=True)][:4]  # precision
# `top` now goes into the prompt — smaller, cleaner, more relevant context.
```

9. Generation: Prompting and Citations

The final stage stitches retrieved passages into a grounded prompt. The system prompt must instruct the model to answer *only* from context and to say "I don't know" when the context is insufficient.

PYTHON

```
SYSTEM = """You are a precise assistant. Answer the question using ONLY the
context passages. If the answer is not in the context, say you don't know.
Cite sources inline as [1], [2] matching the passage numbers."""

def build_prompt(question, passages):
    ctx = "\n\n".join(f"[{i+1}] {p.page_content}" for i, p in enumerate(passages))
    return [
        {"role": "system", "content": SYSTEM},
        {"role": "user", "content": f"Context:\n{ctx}\n\nQuestion: {question}"},
    ]

# With Anthropic's SDK (temperature=0 for factual, deterministic answers):
import anthropic
client = anthropic.Anthropic()
msgs = build_prompt("How many vacation days do new hires get?", top_passages)
resp = client.messages.create(
    model="claude-opus-4-8",   # current model id as of 2026
    max_tokens=512, temperature=0,
    system=msgs[0]["content"],
    messages=[msgs[1]],
)
print(resp.content[0].text)
```

✦ **Prompt-caching & context order**

Put stable, large context (retrieved passages, long system prompt) at the front so it can be cached across turns; keep the volatile question last. Models attend most reliably to the start and end of context — bury nothing important in the middle ("lost in the middle").

10. Evaluating a RAG System

You cannot improve what you don't measure. RAG evaluation splits into **retrieval quality** and **generation quality**, and the two failure surfaces must be measured separately so you know which stage to fix.

RETRIEVAL METRICS

Recall@k: fraction of relevant docs found in top-k. **Precision@k**: fraction of top-k that are relevant. **MRR / nDCG**: reward putting the right doc high.

GENERATION METRICS

Faithfulness: every claim is supported by context (no hallucination). **Answer relevance**: the answer addresses the question. **Context relevance**: retrieved passages were on-topic.

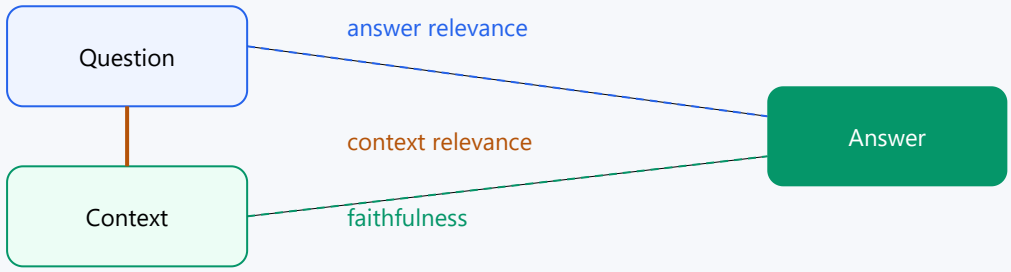


Fig 6 — The RAG evaluation triad: each metric ties two of {question, context, answer} together.

PYTHON

```python
# Sketch of an LLM-as-judge faithfulness check (the idea behind RAGAS)
JUDGE = """Given CONTEXT and an ANSWER, list each factual claim in the ANSWER.
For each, output SUPPORTED if the CONTEXT entails it, else UNSUPPORTED.
Faithfulness = supported_claims / total_claims."""

def faithfulness(context, answer, client):
    r = client.messages.create(
        model="claude-opus-4-8", max_tokens=400, temperature=0,
        system=JUDGE,
        messages=[{"role": "user",
                   "content": f"CONTEXT:\n{context}\n\nANSWER:\n{answer}"}],
    )
    return r.content[0].text   # parse SUPPORTED/UNSUPPORTED counts
```

Build a golden set

Curate 50–200 (question, ideal-answer, relevant-doc-ids) triples. Run it on every change to catch regressions. Automated LLM-judge scores are directional; the golden set is your ground truth.

11. Common Failure Modes

RETRIEVAL MISS

The right passage isn't in top-k. Causes: bad chunking, domain-mismatched embeddings, no hybrid search for exact terms. Fix: wider k + rerank, hybrid, better embeddings.

LOST IN THE MIDDLE

The right passage *is* retrieved but buried in a long context and ignored. Fix: rerank to fewer passages, reorder, shorten context.

HALLUCINATION DESPITE CONTEXT

Model answers from prior knowledge, not the passages. Fix: strict "only from context" prompt, temperature 0, faithfulness eval.

STALE INDEX

Docs updated but vectors not re-embedded. Fix: incremental upserts keyed by content hash + document versioning.

Silent quality decay

A RAG system rarely errors loudly — it just quietly returns worse answers as the corpus grows or embeddings drift. Without continuous evaluation on a golden set you will not notice until users complain.

12. Advanced RAG Patterns

QUERY REWRITING / HYDE

Rewrite terse queries, or generate a hypothetical answer and embed *that* to retrieve — bridges the vocabulary gap between questions and documents.

MULTI-QUERY FUSION

Generate several paraphrases of the query, retrieve for each, and fuse with RRF for higher recall.

SELF / CORRECTIVE RAG

The model grades retrieved passages; if they're weak it re-retrieves, reformulates, or falls back to web search before answering.

AGENTIC RAG

Retrieval becomes a tool the LLM calls on demand, possibly multiple times, mid-reasoning — good for multi-hop questions.

GRAPHRAG

Build a knowledge graph of entities/relations; retrieve subgraphs and community summaries for global "connect the dots" questions.

CONTEXTUAL RETRIEVAL

Prepend an LLM-generated document-level summary to each chunk before embedding, so isolated chunks retain global context.

PYTHON

```python
# HyDE: retrieve using a hypothetical answer's embedding, not the raw question
def hyde_retrieve(question, llm, embedder, store):
    hypo = llm(f"Write a short factual passage answering: {question}")
    qvec = embedder.encode(hypo, normalize_embeddings=True)  # embed the answer
    return store.similarity_search_by_vector(qvec, k=8)      # denser match
```

Mini worked example: a support-bot retrieval decision



Pure dense search might rank a semantically similar but wrong error higher; BM25 anchors on the exact code `E-4021`, and the reranker confirms the passage actually explains that code.

13. Cheat-Sheet Reference

Concept	One-liner	Typical value
Chunk size	Retrievable unit size	200–400 tokens
Overlap	Shared text between chunks	10–20%
Retrieval k	Candidates before rerank	20–50
Kept after rerank	Passages sent to LLM	3–5
Similarity metric	Vector closeness	cosine
Fusion	Combine dense + sparse	RRF (k=60)
Generation temp	Randomness	0 for factual

RAG VS FINE-TUNING VS LONG-CONTEXT

Need	Best tool	Why
Fresh / private facts	RAG	Update the index, cite sources
New behaviour / style / format	Fine-tuning	Changes model weights
One-off analysis of a single doc	Long-context	Just paste it; no infra
Huge, changing corpus	RAG	Can't fit in context; must retrieve

14. Best Practices and Pitfalls

✓ **Do**

Store rich metadata and always return citations. Evaluate retrieval and generation separately. Start simple (fixed chunks + dense) and add hybrid/rerank only when metrics justify it.

✦ **Do**

Version your index and embedding model together; a model change requires a full re-embed. Log every query, its retrieved ids, and the answer for offline analysis.

⚠ **Avoid**

Chunking blindly by character count across tables/code. Mixing embedding models between index and query. Sending 30 passages hoping "more context helps" — it hurts precision and cost.

✕ **Never**

Trust an ungrounded answer just because it cites a number. Verify citations map to real passages, and keep temperature at 0 for factual retrieval.

15. Interview Questions & Answers

What problem does RAG solve that a bigger LLM does not?

Access to fresh, private, or high-volume knowledge with source attribution. Scaling parameters improves reasoning but not knowledge of data the model never saw, and gives no citations or freshness. RAG externalises knowledge into an updatable index.

Why chunk documents at all — why not embed whole files?

Embeddings compress a whole text into one vector; a long document's vector becomes a blurry average that matches nothing precisely, and you'd waste context tokens returning irrelevant sections. Chunking gives precise, focused units that embed and retrieve cleanly.

Bi-encoder vs cross-encoder — when do you use each?

Bi-encoders encode query and doc separately, so document vectors are precomputed and searched fast — use for first-stage retrieval over the full corpus. Cross-encoders process query+doc jointly with cross-attention, far more accurate but O(candidates) forward passes — use to rerank a short candidate list.

Why add keyword (BM25) search if you already have embeddings?

Dense retrieval can miss exact tokens — error codes, IDs, rare proper nouns — because they get smeared into semantic space. BM25 matches them exactly. Hybrid search via RRF gives you both semantic recall and lexical precision.

How do you measure whether a RAG system is good?

Separate retrieval metrics (recall@k, precision@k, MRR/nDCG against labelled relevant docs) from generation metrics (faithfulness, answer relevance, context relevance). Maintain a golden set of question/answer/relevant-doc triples and track it on every change.

What is "lost in the middle" and how do you mitigate it?

LLMs attend most reliably to the beginning and end of long contexts, so a relevant passage placed in the middle can be ignored. Mitigate by reranking down to few passages, ordering the strongest first/last, and keeping total context tight.

The model still hallucinates even with good retrieval. What now?

Tighten the prompt to answer only from context and to say "I don't know" otherwise; set temperature 0; run a faithfulness check that flags claims unsupported by context; and verify retrieval actually contained the answer (else it's a retrieval miss, not a generation problem).

Explain HNSW at a high level.

A multi-layer proximity graph. Upper layers have few nodes with long-range links for fast coarse navigation; lower layers are dense. Search starts at the top, greedily hops toward the query, and descends layer by layer, refining to true neighbours at the base — giving log-ish search over millions of vectors.

How would you handle a corpus that updates constantly?

Incremental upserts keyed by a content hash so unchanged chunks are skipped; delete-and-reinsert changed docs; version documents and store timestamps in metadata for recency filtering. Never require a full rebuild for small edits.

When is RAG the wrong choice?

When the task needs new behaviour or output format (fine-tune instead), when the knowledge fits comfortably in a single prompt (just use long context), or when latency budgets can't absorb a retrieval+rerank round trip and the knowledge is small and static.

What is HyDE and why can it help?

Hypothetical Document Embeddings: use the LLM to draft a plausible answer to the query, then embed and retrieve with that draft. It bridges the vocabulary/asymmetry gap between short questions and prose documents, since an answer-shaped text sits closer to real passages in embedding space.

Design a RAG system for 10M internal documents. What are the key decisions?

Ingestion pipeline with robust parsers and structural chunking + metadata; a domain-appropriate embedding model; an ANN store (HNSW) sharded for scale with metadata filtering and access control; hybrid retrieval + cross-encoder rerank; a grounded, citing prompt at temp 0; and continuous evaluation on a golden set with query logging. Discuss cost, latency, freshness (incremental upserts), and security (per-user filtering) trade-offs.

16. Further Learning

- Read the original RAG paper (Lewis et al., 2020) and the "Lost in the Middle" paper for foundational intuition.
- Build the golden-set evaluation harness before optimising — try the RAGAS library for faithfulness/relevance metrics.
- Experiment with a cross-encoder reranker and measure precision@k gains against the dense-only baseline.
- Study advanced patterns: HyDE, multi-query fusion, contextual retrieval, GraphRAG, and agentic/corrective RAG.
- Learn one production vector store deeply (FAISS internals, or a managed store's filtering/scaling model) and one framework (LangChain or LlamaIndex).
- Practise the system-design interview: sketch the offline vs online paths, name every knob, and justify cost/latency/freshness trade-offs aloud.